

Iterative Learning Control Model Error Compensation

Sanghyun Kim

Kyung Hee University
RCI Lab

Motivation: Model Errors

Reality:

- Models are approximate
- Constraints are tight
- Safety requirements are strict

Motivation: Model Errors

Reality:

- Models are approximate
- Constraints are tight
- Safety requirements are strict

Feedback (LQR/MPC) helps, but sometimes not enough.

Ways to Handle Model Errors

1) Parameter Estimation (Gray-box)

- Fit masses, inertias, lengths
- Sample efficient
- Assumes correct structure

Ways to Handle Model Errors

1) Parameter Estimation (Gray-box)

- Fit masses, inertias, lengths
- Sample efficient
- Assumes correct structure

2) Learn Model (Black-box)

- Neural dynamics model
- Flexible but data hungry

3) Learn Policy (Model-free RL)

- No model assumptions
- Requires many rollouts
- Weak generalization

3) Learn Policy (Model-free RL)

- No model assumptions
- Requires many rollouts
- Weak generalization

4) Transfer / ILC

- Start from nominal trajectory
- Improve using real data
- Very sample efficient

Basic Idea of ILC

Given reference trajectory (x^*, u^*) .

Controller form:

$$u_k = u_k^* - K_k(x_k - x_k^*)$$

We only update feedforward term u_k^* .

Trajectory Optimization View

Cost:

$$J = \sum (x_k - x_k^*)^T Q (x_k - x_k^*) + (u_k - u_k^*)^T R (u_k - u_k^*)$$

Subject to:

$$x_{k+1} = f(x_k, u_k)$$

KKT System Structure

Linearized optimality conditions:

$$H\Delta z = -g$$

where

$$z = (x, u)$$

KKT System Structure

Linearized optimality conditions:

$$H\Delta z = -g$$

where

$$z = (x, u)$$

$H \simeq$ Gauss-Newton Hessian

g = gradient of Lagrangian

Key Observation

If we rollout on real system:

$$c(x, u) = 0$$

Constraint residual is zero (for true dynamics).

Key Observation

If we rollout on real system:

$$c(x, u) = 0$$

Constraint residual is zero (for true dynamics).
Thus we obtain RHS vector g from real rollout.

ILC Update Rule

Solve:

$$H\Delta u = -g$$

Update:

$$u^{new} = u^{old} + \Delta u$$

Repeat until convergence.

Connection to Newton's Method

Exact Newton:

$$\nabla^2 J \Delta u = -\nabla J$$

ILC uses approximate Hessian:

$$H \approx \nabla^2 J$$

Connection to Newton's Method

Exact Newton:

$$\nabla^2 J \Delta u = -\nabla J$$

ILC uses approximate Hessian:

$$H \approx \nabla^2 J$$

This is a quasi-Newton method.

Inexact Newton Convergence

If update satisfies:

$$\|r + J\Delta u\| \leq \eta \|r\|$$

with $\eta < 1$,
then convergence is guaranteed.

Inexact Newton Convergence

If update satisfies:

$$\|r + J\Delta u\| \leq \eta \|r\|$$

with $\eta < 1$,
then convergence is guaranteed.
ILC fits into this framework.

1. Compute nominal trajectory (x^*, u^*)
2. Rollout on real system
3. Compute tracking error
4. Linearize nominal model
5. Solve QP / linear system
6. Update feedforward control
7. Repeat

- Uses real dynamics for gradient
- Uses nominal model for curvature
- Sample efficient
- Task specific

ILC vs RL

	ILC	RL
Sample efficiency	High	Low
Generalization	Low	Moderate
Model needed	Yes	No
Convergence speed	Fast	Slow

- Aggressive quadrotor flight
- High-speed manipulation
- Legged locomotion refinement
- Iterative MPC

Summary

- ILC = structured policy update
- Equivalent to quasi-Newton step
- Very sample efficient
- Powerful when nominal model is decent